THE PROOF BOUNDARY FOR NVIDIA EXECUTION

BY ANONYMOUS AUTHOR,

Roop can generate and launch PTX on NVIDIA hardware. A runtime test passed on an NVIDIA GeForce RTX 4090 with driver 595.91.07, covering kernel launch and writable-buffer copyback. This is execution evidence, not a proof that CUDA execution refines Roop Lean semantics. We state the missing source, compiler, and backend claims, then give a narrow route from an integer additive kernel to a refinement theorem. The separation prevents a hardware regression test from being overstated as compiler verification.

1. Three claims. Let P be a well-formed Roop program, $\mathrm{Sem}(P)$ its Lean state transition, and $K(P)$ the emitted kernel. The source theorem has the form

$$\mathrm{Sem}(\mathrm{inv}(P))(\mathrm{Sem}(P)(s)) = s, \tag{1}$$

with the language guards, stack state, and supported effects included in the model.

A compiler theorem requires a correspondence relation

$$K(P) \sqsubseteq \mathrm{Sem}(P). \tag{2}$$

A backend theorem relates a PTX launch and device memory to the kernel semantics:

$$\mathrm{launch}_{\mathrm{PTX}}(K(P), m) = m' \implies m' \sim \mathrm{Sem}(P)(m). \tag{3}$$

Only the conjunction of these statements justifies a claim about the running GPU program.

2. Current evidence. The CUDA regression test passed on the RTX 4090 configuration recorded in the repository benchmark record. It tests PTX launch and host-visible writable-buffer copyback. It does not cover arbitrary kernels, block shapes, alias patterns, memory ordering, launch failures, or floating-point behavior.

The repository contains no CUDA throughput comparison. It would therefore be inaccurate to call the backend competitive. A performance report must name the device, driver, CUDA toolchain, tensor shape, element type, operation count, launch configuration, warm-up policy, repetitions, variance, data-residency model, and baseline.

3. A narrow theorem. Begin with an integer additive kernel over ‘BitVec 64‘ buffers. Give it the functional specification

$$p \leftarrow p + f(q), \tag{4}$$

where q remains unchanged. Prove that the compiler emits the kernel only when its index and alias conditions hold. Then validate or prove a PTX refinement for that restricted launch form. The inverse kernel must compute $p \leftarrow p - f(q)$ from the same preserved q.

The certificate calculus provides the matching source-level theorem. It proves exact modular round trips but does not identify a generated kernel with its specification. Hardware tests remain useful regression tests. They do not close the compiler or backend proof boundary.

MSC2020 subject classifications. Primary 68N20.

Keywords and phrases. CUDA, compiler verification, reversible computation.